

Documentación y Análisis del Proyecto

Calculadora Modular e Interactiva

1. Introducción

Este proyecto consiste en el desarrollo de una aplicación de software modular en JavaScript para la ejecución de cálculos matemáticos, secuencias lógicas y aplicaciones de ingeniería eléctrica (Ley de Ohm), entre otras.

El sistema fue diseñado con un enfoque híbrido, permitiendo la interacción del usuario tanto a través de una interfaz gráfica web dinámica (HTML5/Bootstrap) como mediante una interfaz de comandos interactiva en consola (*Modo prompt* con ***prompt*** y ***alert***).

2. Objetivos del Proyecto

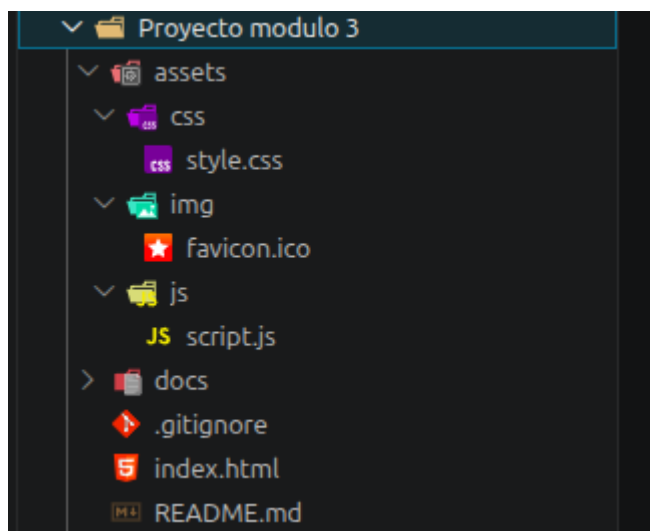
Operaciones matemáticas	●
Condicionales (if, switch)	●
Bucles (for, while)	●
Funciones	●
Arreglos	●
Objetos	●
Ejecutar en consola	●
Prompt js	●
Validaciones	●
Código fuente en Github	●
Archivo README.md	●
Capturas de pantalla	●
Documentación y análisis	●

3. Arquitectura del Código y Análisis Técnico

Al inicio no entendí bien el requerimiento específico del proyecto, por lo que había avanzado en algo completamente diferente. Sin embargo, decidí mantenerlo y en base a lo que ya tenía, agregué el formato solicitado.

El proyecto fue desarrollado utilizando una arquitectura modular basada en separación de responsabilidades. La interfaz gráfica, la lógica de procesamiento y la gestión de eventos fueron divididas en distintas secciones para mejorar la mantenibilidad y escalabilidad del sistema.

A. Estructura General del Proyecto



El proyecto fue ordenado según las indicaciones del curso con cada archivo en sus respectivos directorios.

Como se muestra en la imagen, existe un directorio assets donde se encuentran los archivos necesarios para los estilos y la dinámica de la web en sus respectivos subdirectorios. Además incluí un directorio img con favicon.ico sólo para no ver el error de su ausencia en la consola del navegador.

También podemos ver el directorio docs, donde se encuentran las capturas del programa web y esta documentación.

En la raíz, los archivos de .gitignore, README.md y el principal. index.html el cual contiene la estructura de la página web.

B. Estructura de componentes

A continuación detallo la estructura de componentes:

- Interfaz HTML organizada mediante Bootstrap.
- Formularios independientes por categoría de operación.
- Procesamiento centralizado en JavaScript.
- Consola visual para resultados en el HTML.
- Sistema reutilizable de eventos.
- Salida en consola de navegador.
- Ingreso por 3 vías: formularios, botón prompt y menú prompt.

C. Funcionalidades

Operaciones básicas

Incluye:

- suma
- resta
- multiplicación
- división

Operaciones avanzadas

Incluye:

- potencia
- raíz
- módulo
- porcentajes

Ley de Ohm y Watt

Incluye:

- cálculo de watts
- amperaje
- voltaje
- resistencia

Operaciones especiales

Incluye:

- correlativos
 - secuencia Fibonacci
-

4. Análisis Técnico

Modularidad

El sistema fue diseñado mediante módulos funcionales independientes, permitiendo separar cada grupo de operaciones matemáticas según su propósito. Esto facilita futuras ampliaciones sin alterar la lógica principal del proyecto.

Manejo de eventos

Se utilizó programación basada en eventos mediante formularios HTML y JavaScript, permitiendo capturar dinámicamente las acciones del usuario y procesar los cálculos en tiempo real.

Validación de datos

El sistema valida la entrada de datos antes de ejecutar las operaciones matemáticas, evitando errores de ejecución y asegurando que los cálculos se realicen únicamente con valores válidos.

Interfaz responsive

Se implementó Bootstrap para lograr compatibilidad responsive en distintos tamaños de pantalla, mejorando la experiencia de usuario tanto en escritorio como dispositivos móviles.

5. Experiencia de usuario

La interfaz fue diseñada priorizando simplicidad y claridad visual, utilizando tarjetas independientes y controles intuitivos para facilitar el uso del sistema.

Se incluyó un div simulador de consola para que no fuera necesario abrir la consola del navegador.

6. Control de Errores

El sistema mitiga fallos críticos mediante validaciones previas al cálculo:

- Entradas insuficientes: Validación de longitud mínima del arreglo (**`length < 2`**) para operaciones binarias.
- Datos Corruptos (**`NaN`**): Uso de **`.some(isNaN)`** para interceptar texto o letras camufladas entre los números.
- División por Cero e Indeterminación: Implementación de una bandera de estado global sincronizada (**`esCero`**) que detiene los motores de renderizado gráfico de tablas HTML si el denominador evaluado es **`0`**.
- Se verifica que se haya seleccionado una operación, que la entrada contenga números, que no solo contenga comas, entre otras validaciones.

7. Conocimientos Adquiridos

Durante la creación del proyecto, fue necesario revisar webs especializadas en javascript y con ayuda de diferentes IA orientadas a programación para poder encontrar soluciones a lo que necesitaba hacer. Es por eso que detallo a grandes rasgos qué conocimientos adquirí en la realización del código y el por qué utilicé esos extra.

En el código existen 2 **`document.addEventListener`**, esto es porque al darme cuenta de que había empezado a hacer el proyecto basado en formularios, había decidido usar el llamado de submit. Sin embargo, tuve que agregar un nuevo **`addEvenListener`** para escuchar un nuevo botón que abriría un prompt solicitando números. Con esto, tuve que buscar la forma de escuchar sólo ese botón y de ese formulario específico. Una de las formas que me funcionó fue que el **`Listener`** escuchara al botón y revisara si contenía el class btn-prompt con la siguiente línea: **`evento.target.classList.contains("btn-prompt")`**.

Si bien es cierto, aprendí muchas cosas a medida que avanzaba el proyecto, otra cosa interesante aprendida fue el uso de **`Promise`**.

Esto fue sólo un extra para que cierta salida no fuera tan repentina y tuviera cierta animación. Las líneas en cuestión son:

```
const esperar = ms => new Promise(res => setTimeout(res, ms));  
await esperar(500);
```

Estas líneas (además de necesitar incluir async en ciertas funciones), me ayudaron a hacer la "magia" de que en las funciones "números correlativos" y "fibonacci", se pudieran ver los números en consola uno por uno y no de golpe.

5. Conclusión

El desarrollo de esta calculadora me permitió consolidar conceptos aprendidos y ponerlos en práctica. Además, me permitió aprender más cosas y adentrarme en el mundo de javascripts al intentar encontrar solución a las distintas dificultades que se me presentaron a medida que construía la aplicación.